Recommandations (enjeux, constats ou lacunes à résoudre/solutionner)

- Prérequis transversal — appui explicite de la direction et des partenaires R&D
- Ajustement de la gouvernance Jira et Xray
  - Conséquences si la gouvernance Jira et Xray n'est pas uniformisée
  - Consolidation des projets Jira par produit — un seul espace/projet par produit
    - Pourquoi agir maintenant, même si le produit est en maintenance ou appelé à être remplacé
    - Raisons de ne pas attendre
    - Règles de consolidation recommandées
  - Distinction entre Jira et Xray — livraison vs définition produit
  - Règles d'utilisation recommandées
  - Shift left de l'acceptation dans le flux produit
- Ajustement du centre d'excellence
  - Conséquences si le centre d'excellence n'est pas explicitement structuré
  - Mandat recommandé du centre d'excellence
  - Formation continue comme pilier du centre d'excellence
- Ajustement du positionnement de la qualité dans l'organisation
  - Conséquences si la qualité n'est pas repositionnée comme catalyseur intégré
- Ajustement de la responsabilité produit complète (SENSE)
  - Conséquences si les rôles Produit et Livraison ne sont pas explicitement assignés
- Ajustement de la voix du client (Customer Voice)
  - Conséquences si la voix du client n'est pas structurée
  - Mécanismes recommandés
- Ajustement du contrat qualité
  - Conséquences si le contrat qualité n'est pas formalisé
- Ajustement de la centralisation de la documentation produit et des spécifications
  - Conséquences si la documentation produit et les spécifications ne sont pas centralisées
- Ajustement du modèle de versionnement et du découpage des livraisons
  - Conséquences si le modèle de versionnement et le découpage des livraisons ne sont pas uniformisés
  - Niveaux de version recommandés
  - Alignement entre découpage Jira et versionnement
  - Clarification importante — définition de FEATURE
  - Règles de granularité recommandées
  - Piège classique — l'illusion de la livraison rapide
  - Bénéfices attendus
- Usage structuré de l'IA dans le SDLC
- Automatisation des validations et couverture ciblée
  - Architecture de modélisation et d'automatisation — AltWalker + pytest + Robot Framework + Xray
    - Pourquoi cette architecture
- Ajustement de la gouvernance des rôles et du pouvoir décisionnel
  - Conséquences si les rôles décisionnels ne sont pas différenciés
  - Principe fondamental — tâches et responsabilités d'abord, rôles ensuite
  - Recommandation — privilégier les rôles catalyseurs-cadres au décisionnel
  - Test de nécessité d'un rôle
- Ajustement de la posture qualité dans les équipes de développement
  - Conséquences si la qualité reste portée par un rôle QA fourre-tout
  - Questions de réflexion — réponses recommandées
- Carte de priorisation et dépendances entre recommandations

Cette page consolide les recommandations formulées dans le cadre de la mission qualité (Mission Faucon). Chaque recommandation répond à un enjeu, un constat ou une lacune observée lors de l'analyse des pratiques de réalisation logicielle chez OPAL-RT.

Les recommandations ne sont pas indépendantes : elles forment un système. La consolidation des projets Jira est un prérequis structurel; la gouvernance Jira/Xray, le contrat qualité, le versionnement et la documentation produit en dépendent; le repositionnement de la qualité, le centre d'excellence et la voix du client soutiennent l'ensemble.

L'ordre de lecture suit une logique de dépendances : les fondations structurelles d'abord, les pratiques ensuite, les mécanismes de pérennité à la fin.

⚠ **Prérequis transversal — appui explicite de la direction et des partenaires R&D**

Les recommandations de cette page touchent des enjeux structurels qui dépassent la capacité d'une équipe ou d'une direction fonctionnelle à les régler seule. Leur mise en œuvre requiert un **appui explicite, complet et soutenu de la direction**, ainsi qu'un **GO organisationnel clair** pour les recommandations bloquantes.

Cet appui est particulièrement nécessaire pour : **livrer plus petit par version**, **uniformiser Jira/Xray**, **consolider les espaces/projets par produit**, **endosser et signer le contrat qualité**, **officialiser les accords RACI**, **documenter le produit**, **structurer la couverture Xray**, et **avancer sur la modélisation et l'automatisation**.

Sans ce GO, les recommandations risquent de rester des documents Confluence bien rédigés mais non appliqués. Les équipes locales feront ce qu'elles peuvent dans leur périmètre, mais les changements structurels — consolidation Jira, contrat qualité signé, accords RACI, versionnement unifié, couverture Xray globale — resteront bloqués.

Recommandation : obtenir un GO formel de la direction pour les recommandations urgentes ou structurelles. Ce GO doit inclure un engagement sur les ressources, le calendrier, le mandat et le soutien organisationnel nécessaires. Les recommandations qui restent sans GO explicite doivent être documentées comme **risques acceptés** plutôt que comme plans en attente.

Point de vigilance : l'appui de la direction ne signifie pas que la direction fait le travail. Cela signifie qu'elle **autorise, protège, priorise et assume** les changements nécessaires. Les équipes exécutent; la direction débloque et soutient.

Ajustement de la gouvernance Jira et Xray

L'utilisation de Jira et Xray n'est ni uniforme ni cohérente entre les équipes; la frontière entre la définition du produit, les spécifications, les tests et la livraison doit être clarifiée.

Note terminologique. Cette page formule des recommandations d'ajustement organisationnel et d'outillage. Les définitions de référence des termes utilisés ici — Jira, Xray, EPIC, FEATURE, STORY, BUG, spécification, validation, acceptation, DoR, DoD, qualité, qualimétrie, rôle, responsabilité, SENSE, FORGE, CARE, PM, PO, CdL et RACI — sont maintenues dans le Lexique / Terminologies. Les descriptions locales précisent l'usage recommandé dans le contexte de ces ajustements et ne remplacent pas le lexique ni la matrice RACI.

> ⚠ **Conséquences si la gouvernance Jira et Xray n'est pas uniformisée**
>
> Sans règles communes d'utilisation de Jira et Xray, les équipes mélangent le découpage de livraison, la documentation produit, les spécifications et les preuves qualité. Le risque principal est de transformer Jira en référentiel produit implicite, alors que Jira devrait surtout représenter le montage de la livraison et le suivi des tâches à faire.

Consolidation des projets Jira par produit — un seul espace/projet par produit

Pour un même produit comme **RT-LAB**, les équipes ne devraient pas travailler dans des projets Jira séparés. La situation actuelle — un espace ou projet par équipe pour le même produit — fragmente le backlog, les versions, les releases, la couverture Xray et la documentation produit. Elle rend la gouvernance pratiquement impossible, car chaque équipe finit par gérer sa propre structure, ses propres conventions et ses propres preuves qualité.

> ⚠ **Urgence :** cette consolidation doit être traitée rapidement. Tant que RT-LAB reste réparti dans plusieurs projets Jira d'équipe, les autres recommandations de gouvernance — hiérarchie EPIC → FEATURE → STORY/BUG, versions communes, couverture Xray, documentation produit, métriques et tableaux de bord — ne pourront pas être appliquées correctement.

Élément absent ou flou	Ce qui devient flou	Conséquences possibles	Symptômes observables
Projet Jira unique par produit	Vue consolidée du backlog produit, de la hiérarchie EPIC → FEATURE →	Chaque équipe gère son propre backlog, ses propres workflows, ses propres conventions	Requêtes JQL complexes ou impossibles, métriques non comparables,

	STORY/BUG et de l'avancement global.	et ses propres priorités, sans vision produit commune.	impossibilité de voir l'état réel de RT-LAB en un coup d'œil.
Versions et releases communes	Alignement entre les versions livrées, les branches, les pipelines, les notes de version et les engagements produit.	Une même release produit peut être représentée par plusieurs versions Jira incompatibles ou nommées différemment selon les équipes.	Versions en double, release notes fragmentées, confusion sur ce qui est réellement livré, traçabilité version → EPIC → FEATURE → STORY/BUG brisée.
Hiérarchie EPIC → FEATURE → STORY/BUG partagée	Rattachement de chaque item de réalisation à son contexte parent produit.	Les EPIC, FEATURE, STORY et BUG existent en parallèle dans plusieurs projets sans structure commune; la hiérarchie obligatoire devient difficile à appliquer.	Items orphelins, duplication de FEATURE ou d'EPIC, perte du contexte produit, difficulté à relier la valeur affaire à la réalisation.
Couverture et traçabilité Xray unifiées	Lien entre spécifications IF... THEN..., tests Xray, résultats, couverture et preuve de validation au niveau produit.	Les plans de validation, test sets, exécutions et résultats sont dispersés entre projets, ce qui empêche une couverture produit consolidée.	Couverture partielle par équipe, duplication de tests, impossibilité de démontrer la couverture globale d'une release RT-LAB.
Documentation produit centralisée	Source unique de vérité pour les spécifications, décisions, modèles, validations et preuves produit.	Chaque équipe documente selon son propre espace ou projet; les décisions se dispersent et les spécifications peuvent se contredire.	Documentation dispersée, décisions difficiles à retrouver, revalidation fréquente de sujets déjà tranchés ailleurs.

Tableaux d'équipe dans un même projet	Différence entre isoler une vue d'équipe et fragmenter le produit dans plusieurs projets Jira.	Les équipes croient devoir créer un projet séparé pour avoir leur propre tableau, alors que Jira permet plusieurs tableaux filtrés dans un même projet.	Création de projets inutiles, multiplication des conventions, duplication des items, incapacité à travailler efficacement ensemble.

Effet combiné : lorsque chaque équipe possède son propre projet Jira pour RT-LAB, tout ce qui devrait être commun se fragmente : backlog, hiérarchie, versions, releases, couverture Xray, documentation, métriques et gouvernance. Les équipes dupliquent les items, perdent la traçabilité et ne peuvent pas exploiter Jira et Xray de façon optimale. Le problème n'est pas seulement administratif : il empêche concrètement les équipes de travailler ensemble sur le même produit.

Recommandation urgente : consolider RT-LAB dans **un seul projet Jira / espace produit** partagé par toutes les équipes. Les équipes doivent conserver leur autonomie opérationnelle au moyen de **tableaux Jira distincts dans le même projet**, basés sur des filtres, composants, labels, équipes, sprints ou vues dédiées. C'est la façon recommandée de travailler : un produit commun, plusieurs tableaux d'équipe.

⚠ **Pourquoi agir maintenant, même si le produit est en maintenance ou appelé à être remplacé**

Il peut sembler contre-intuitif d'investir dans la consolidation Jira/Xray et dans la documentation produit lorsqu'un produit est déjà en maintenance, en stabilisation ou susceptible d'être remplacé plus tard. En réalité, c'est précisément le bon moment pour le faire, et l'urgence est encore plus grande.

Un produit en maintenance ne signifie pas un produit sans enjeu. C'est souvent à ce moment que l'organisation doit réagir rapidement aux incidents, préparer la fin de vie, réduire l'insatisfaction client et comprendre ce qui doit absolument être évité dans le produit successeur. Plus la réaction est lente, plus l'insatisfaction augmente rapidement, parfois de façon exponentielle.

Raisons de ne pas attendre

- **Planifier correctement la fin de vie du produit.** Sans documentation produit, versions communes, couverture Xray et traçabilité Jira, il devient difficile de décider ce qui doit être maintenu, corrigé, déprécié, communiqué ou transféré vers le produit successeur.

- **Comprendre les échecs avant de reconstruire.** Consolider Jira/Xray permet d'analyser les défauts récurrents, les zones instables, les décisions passées, les lacunes de validation et les irritants clients. Sans cette lecture, le nouveau produit risque de répéter les erreurs du passé.
- **Réagir plus vite en maintenance et stabilisation.** Quand un produit est en maintenance, les demandes sont souvent liées à des incidents, correctifs, régressions ou enjeux clients urgents. Une structure Jira/Xray fragmentée ralentit l'analyse, la priorisation, la reproduction, la validation et la livraison des correctifs.
- **Réduire l'insatisfaction client.** Chaque délai supplémentaire pour comprendre, prioriser et corriger un problème augmente la frustration. Plus le cycle de réaction est long, plus l'insatisfaction grimpe rapidement et plus la confiance envers le produit et l'organisation diminue.
- **Préparer la transition vers le produit successeur.** La documentation, les tests Xray, les décisions et les métriques de maintenance deviennent des intrants essentiels pour HYPER-LAB ou tout autre produit successeur : ce qu'il faut conserver, retirer, simplifier, corriger ou repenser.
- **Éviter de transférer le désordre organisationnel.** Si Jira, Xray, les versions, les espaces et les preuves qualité restent fragmentés dans le produit actuel, le même modèle risque d'être reproduit dans le nouveau produit. La consolidation actuelle sert aussi de répétition et de fondation pour la suite.

Conclusion : il ne faut pas attendre le remplacement éventuel du produit pour redresser Jira, Xray et la documentation. Pour un produit en maintenance, la consolidation est un accélérateur de stabilisation, un outil de fin de vie, une source d'apprentissage pour le produit successeur et un moyen concret de réduire l'insatisfaction client maintenant.

Règles de consolidation recommandées

- **Un projet Jira = un produit.** Pour RT-LAB, utiliser un seul projet Jira produit, pas un projet par équipe, composant ou discipline.
- **Un tableau par équipe.** Chaque équipe peut avoir son propre board, ses filtres, ses sprints et ses vues, tout en travaillant dans le même backlog produit.
- **Une hiérarchie unique.** Appliquer la structure EPIC → FEATURE → STORY/BUG de façon uniforme pour toutes les équipes.
- **Des versions et releases communes.** Toutes les équipes utilisent les mêmes versions Jira, alignées avec le modèle de versionnement officiel.
- **Une couverture Xray consolidée.** Les plans de validation, tests, test sets, exécutions et résultats doivent permettre une vue produit globale.

- **Une documentation produit centralisée.** Les spécifications, décisions, modèles et preuves de validation doivent être reliés au même produit et non dispersés par équipe.
- **Des composants et filtres pour segmenter.** Utiliser les composants, labels, champs d'équipe et filtres Jira pour distinguer le travail par équipe ou module, sans fragmenter le projet produit.

Point de vigilance : cette consolidation est un prérequis structurel. Sans un projet Jira produit unique pour RT-LAB, les recommandations sur la hiérarchie obligatoire, le versionnement, la couverture Xray, le contrat qualité, les métriques et la documentation resteront théoriques ou seront appliquées de façon incohérente.

Distinction entre Jira et Xray — livraison vs définition produit

Au-delà de la consolidation des projets, il faut clarifier le rôle respectif de Jira et de Xray. Sans cette distinction, même un projet Jira unique par produit deviendra un mélange confus de livraison, documentation, spécifications et preuves qualité.

Élément absent ou flou	Ce qui devient flou	Conséquences possibles	Symptômes observables
Rôle respectif de Jira et Xray	Distinction entre livraison du produit et définition du produit.	Jira devient un mélange de backlog, documentation, spécifications, décisions, tâches, bugs et preuves qualité.	Informations dupliquées ou contradictoires, difficulté à savoir où se trouve la vérité produit, débats sur l'outil plutôt que sur le besoin.
Jira comme système de livraison	Découpage, montage, planification et suivi des livraisons et des tâches à faire.	Les items Jira sont utilisés pour définir le produit en profondeur, ce qui rend la documentation fragmentée et difficile à maintenir.	Stories trop longues, champs Jira utilisés comme documents, décisions importantes perdues dans des commentaires ou des tickets fermés.
Xray comme référentiel de définition produit testable	Utilisation des tests, test sets et spécifications Xray comme représentation	Les spécifications ne sont pas testables ou ne sont pas reliées aux preuves de	Validations créées tard, couverture difficile à démontrer, spécifications non

	vérifiable du produit attendu.	validation; la qualité devient interprétative.	liées aux validations, ambiguïté sur ce qui doit être validé.
Uniformité des types Jira	Quand utiliser EPIC, FEATURE, STORY, BUG ou d'autres types.	Chaque équipe invente sa propre taxonomie, ce qui complique la traçabilité, les métriques et la gouvernance portfolio.	Types personnalisés inutiles, requêtes et tableaux incohérents, difficulté à comparer les équipes ou les versions.
Hiérarchie obligatoire	Relation parent-enfant entre valeur affaire, valeur produit et réalisation.	Une STORY seule, une FEATURE seule ou un BUG orphelin perd son contexte et affaiblit la traçabilité.	Items difficiles à prioriser, bugs sans rattachement produit, stories sans intention claire, reporting incomplet.
Flux produit commun	États attendus pour faire passer un besoin du backlog produit jusqu'à la confirmation et la terminaison.	Le travail avance sans compréhension commune du niveau de maturité : à faire, élicité, défini, planifié, implémenté, diffusé ou confirmé.	Items déplacés trop tôt, stories développées sans définition claire, validation tardive, rework fréquent.

Effet combiné : lorsque Jira et Xray ne sont pas gouvernés comme deux outils complémentaires, l'organisation perd la séparation essentielle entre **définir le produit** et **livrer le produit**. Jira devient trop lourd comme documentation produit, Xray est sous-utilisé comme spécification testable, et les équipes se retrouvent avec une traçabilité incomplète entre besoin, livraison, validation et preuve qualité.

Recommandation : établir une règle simple et explicite : **Xray = définition du produit** et **Jira = livraison du produit**.

- **Jira** doit être utilisé comme système de découpage, de montage, de planification et de suivi de la livraison : EPIC, FEATURE, STORY, BUG, versions, dépendances, tâches à faire et progression du travail.

- **Jira ne devrait pas devenir le référentiel principal de documentation produit** ni l'endroit où l'on définit en détail le produit. Il peut référencer la documentation et les spécifications, mais ne devrait pas les remplacer.
- **Xray** peut faire sens comme référentiel de documentation produit lorsque les tests, test sets et validations représentent les spécifications vérifiables du produit.
- **Les spécifications devraient être exprimées comme tests Xray** lorsque le besoin peut être formulé en comportements attendus, spécifications IF… THEN… et preuves de validation.

Règles d'utilisation recommandées

- **Hiérarchie obligatoire :** EPIC = parent de FEATURE, et FEATURE = parent de STORY ou BUG. Une STORY seule, une FEATURE seule ou un BUG orphelin ne devrait pas exister; chaque item de réalisation doit être rattaché à son contexte parent afin de préserver la traçabilité entre valeur affaire, valeur produit et livraison.
- Une seule EPIC par version majeure.
- Une seule FEATURE par version mineure.
- Un seul BUG par version patch; normalement, une STORY n'a pas de version sauf en mode micro-livraison, où elle peut alors être associée à une version patch comme un BUG.
- Chaque version doit avoir un but ou une valeur produit précise; ce n'est pas un drop box.
- Éviter la multiplication des types comme request, requis, requirement ou autres variantes. Rester simple : utiliser l'EPIC même si le besoin semble petit au départ; il pourra être fusionné ou redécoupé après l'élicitation.
- Éviter tout autre type non essentiel afin de préserver la lisibilité, la gouvernance et les métriques.
- Tableau produit recommandé : À FAIRE (backlog produit), ÉLICITER, DÉFINIR, PLANIFIER, IMPLÉMENTER, DIFFUSER, CONFIRMER et TERMINÉ.

Point de vigilance : Jira et Xray ne doivent pas se faire compétition. Jira sert à organiser et piloter la livraison; Xray sert à formaliser ce qui doit être vérifié pour confirmer que le produit correspond à ce qui est attendu. Cette distinction réduit la confusion, améliore la traçabilité et permet d'avoir une définition produit réellement testable.

Shift left de l'acceptation dans le flux produit

Chez Opal-RT, dans le produit RT-LAB, les validations d'acceptation sont généralement concentrées après la diffusion. Cette pratique peut être acceptable, mais elle concentre l'effort de confirmation à la fin du cycle et augmente le risque de découvrir tardivement des écarts importants entre le besoin, l'implémentation et la satisfaction attendue.

Il serait préférable d'intégrer une partie de l'acceptation plus tôt dans le flux produit, dès les phases **ÉLICITER**, **DÉFINIR**, **PLANIFIER**, **IMPLÉMENTER**, **DIFFUSER** et **CONFIRMER**. L'objectif n'est pas de tout retester à chaque étape, mais de valider progressivement, avec une approche de type **acceptation continue**, que l'équipe demeure sur la bonne trajectoire avant la confirmation finale.

- **ÉLICITER :** valider que le besoin, les irritants et la valeur attendue sont bien compris par les utilisateurs ou représentants concernés.
- **DÉFINIR :** confirmer que les spécifications IF... THEN..., règles d'affaires et comportements attendus sont testables et alignés avec l'usage réel.
- **PLANIFIER :** vérifier que la portée, les risques, les dépendances et les critères de sortie permettent une validation utilisateur réaliste.
- **IMPLÉMENTER :** faire des validations ciblées sur les comportements clés au fur et à mesure, sans attendre que tout soit terminé.
- **DIFFUSER :** s'assurer que la livraison, la communication, les notes de version et la préparation opérationnelle soutiennent bien l'usage attendu.
- **CONFIRMER :** effectuer la confirmation d'acceptation finale en s'appuyant sur les validations déjà faites, plutôt que de redécouvrir l'ensemble du produit à la fin.

Point de vigilance : cette approche peut donner l'impression de retester, mais l'intention est différente. Il s'agit de confirmer progressivement la direction, de réduire les surprises tardives et d'éviter de concentrer tous les efforts d'acceptation à la fin pour finalement constater que la livraison ne produit pas une satisfaction claire et transparente.

Élément absent ou flou	Ce qui devient flou	Conséquences possibles	Symptômes observables
Validation d'acceptation progressive	À quel moment les parties prenantes confirment que le besoin, les spécifications et la valeur attendue sont toujours alignés.	Les écarts sont découverts seulement après la diffusion, lorsque les options de correction sont plus coûteuses et plus limitées.	Acceptation concentrée à la fin, surprises tardives, rework important, pression élevée juste avant la confirmation finale.
Spécifications d'acceptation par phase	Ce qui doit être validé à ÉLICITER, DÉFINIR, PLANIFIER, IMPLÉMENTER,	Chaque phase peut avancer avec des hypothèses non confirmées, ce qui	Questions utilisateur qui reviennent tard, spécifications d'acceptation

	DIFFUSER et CONFIRMER.	donne une fausse impression de progression.	réinterprétées, décisions de scope revalidées en fin de cycle.
Distinction entre retest et confirmation continue	La différence entre refaire les mêmes tests et confirmer progressivement que la livraison reste sur la bonne trajectoire.	L'approche shift left peut être perçue comme une surcharge inutile plutôt que comme une réduction de risque.	Résistance des équipes, validations sautées, perception de doublon, retour au modèle "tout tester à la fin".
Traçabilité entre acceptation, Xray et décision Go/No-Go	Comment les validations intermédiaires deviennent des preuves utilisables pour la confirmation finale.	Les décisions de livraison reposent sur des impressions ou des validations informelles plutôt que sur des preuves accumulées.	Preuves d'acceptation difficiles à retrouver, validations Xray non liées aux spécifications d'acceptation, Go/No-Go basé sur une vue partielle.

Effet combiné : sans shift left de l'acceptation, l'organisation concentre la validation à un moment où le produit est déjà largement livré. Les équipes peuvent alors découvrir trop tard que la solution ne répond pas entièrement au besoin, même si le développement et les tests techniques semblent terminés. En intégrant des confirmations d'acceptation à chaque phase, la validation devient cumulative, plus légère et plus utile pour protéger la satisfaction réelle.

Recommandation : définir des spécifications d'acceptation simples pour chaque étape du flux produit et les relier aux validations Xray et aux décisions de livraison. La confirmation finale devrait s'appuyer sur les validations déjà accumulées plutôt que repartir de zéro après la diffusion.

Ajustement du centre d'excellence

Les rôles catalyseurs du centre d'excellence ne sont pas explicitement structurés.

⚠ **Conséquences si le centre d'excellence n'est pas explicitement structuré**

Le centre d'excellence ne devrait pas être une équipe qui fait le travail à la place des équipes. Il doit agir comme un **levier de transformation** : coacher, faciliter, outiller,

standardiser et rendre les équipes autonomes. Sans ces rôles catalyseurs, les bonnes intentions du SDLC, du contrat qualité et de la gouvernance restent théoriques.

Élément absent ou flou	Ce qui devient flou	Conséquences possibles	Symptômes observables
Coach qualité (Quality Coach)	Accompagnement des pratiques qualité, Definition of Done, stratégie de test, critères de sortie, culture qualité intégrée à l'équipe.	Retour aux anciennes habitudes de contrôle qualité en fin de chaîne; dépendance à un rôle de QA traditionnel; qualité perçue comme externe à l'équipe.	Tests tardifs, défauts découverts en fin de release, débats sur "qui est responsable de la qualité", critères qualité appliqués de façon variable.
Coach agile (Agile Coach)	Facilitation des rituels, collaboration interéquipes, amélioration continue, gestion des dépendances et maturité agile.	Les cérémonies deviennent mécaniques; les silos persistent; les problèmes de flux ne sont pas traités à la racine.	Daily sans décisions, rétrospectives sans actions, dépendances découvertes trop tard, équipes qui "font de l'agile" sans améliorer leur livraison.
Administrateur des outils (Tooling Administrator)	Configuration et évolution de Jira, Confluence, Xray, SonarQube, tableaux de bord, automatisations et intégrations.	Les processus ne sont pas supportés par les outils; les métriques sont difficiles à extraire; la traçabilité devient manuelle ou incohérente.	Champs Jira incomplets, workflows contournés, tableaux de bord non fiables, duplication d'information entre outils.
Technologies de l'information (Information Technology — IT)	Accès, licences, environnements, sécurité, infrastructure, disponibilité des services et soutien technique transversal.	Les équipes perdent du temps sur des blocages d'accès ou d'environnement; les contraintes sécurité et infrastructure sont découvertes trop tard.	Retards causés par les accès, environnements instables, demandes urgentes en fin de livraison, dépendance à des solutions temporaires.

Personnes DevOps (DevOps Engineers)	Automatisation CI/CD, pipelines, packaging, observabilité, déploiement, reproductibilité des builds et intégration entre développement et opérations.	La livraison reste manuelle, fragile ou dépendante de quelques experts; les risques de déploiement et de rollback augmentent.	Pipelines instables, builds non reproductibles, déploiements manuels, surprises lors du passage vers les opérations.
Gestion du changement (Change Management)	Adoption, communication, formation, impacts organisationnels, préparation des utilisateurs et accompagnement des nouvelles pratiques.	Les changements de processus ou d'outils sont imposés plutôt qu'adoptés; la résistance augmente; les anciennes habitudes reviennent.	Faible adoption, contournement des nouveaux processus, incompréhension des rôles, perception que la transformation est administrative.
Facilitateurs transversaux (Facilitators)	Animation des ateliers, alignement des parties prenantes, documentation des décisions et maintien du dialogue entre SENSE, FORGE et CARE.	Les décisions se prennent en silo; les bons joueurs ne sont pas toujours présents; les décisions ne sont pas suffisamment traçables.	Ateliers sans décision claire, revalidation répétée, incompréhensions entre produit, développement, validation, opérations et support.

Effet combiné : sans centre d'excellence structuré, les équipes doivent inventer seules leurs pratiques, leurs outils et leurs mécanismes de collaboration. Cela crée des écarts entre équipes, affaiblit la standardisation et rend la transformation dépendante de quelques individus plutôt que d'un système durable.

Recommandation : formaliser un centre d'excellence composé de rôles catalyseurs. Ces rôles peuvent être partagés ou à temps partiel, mais leurs responsabilités doivent être explicites. Leur mandat n'est pas de faire à la place des équipes, mais de rendre les équipes capables de livrer avec qualité, autonomie et cohérence.

Mandat recommandé du centre d'excellence

- Définir et faire évoluer les pratiques communes liées au SDLC, à la qualité, à l'agilité, aux outils et à la livraison.
- Accompagner les équipes dans l'adoption des pratiques sans créer de dépendance permanente.
- Faciliter les ateliers transversaux pour éviter les silos entre produit, développement, validation, opérations et support.
- Maintenir les gabarits, tableaux de bord, métriques et mécanismes de traçabilité nécessaires à la gouvernance.
- Identifier les irritants systémiques et proposer des améliorations continues.
- Former, coacher et soutenir les rôles clés afin que la qualité soit intégrée dans le travail quotidien.

Point de vigilance : le centre d'excellence ne doit pas devenir une tour de contrôle ni un goulot d'étranglement. Il doit agir comme un accélérateur, un filet de sécurité et un mécanisme de cohérence organisationnelle.

Formation continue comme pilier du centre d'excellence

Le centre d'excellence doit inclure un programme de **formation continue** comme mécanisme de pérennité organisationnelle. Les détails du parcours, des modules, des ateliers et des sources à utiliser sont maintenus dans la page Formation en continue, qui doit rester la source de vérité pour éviter de dupliquer le contenu ici.

Ajustement du positionnement de la qualité dans l'organisation

La qualité est isolée du développement, des opérations et du portfolio — et la dynamique relationnelle entre R&D et qualité amplifie le problème.

> ⚠ **Conséquences si la qualité n'est pas repositionnée comme catalyseur intégré**
>
> Sans positionnement explicite de la qualité comme pratique intégrée et catalyseur transversal, elle devient un rôle isolé, perçu comme externe, contraignant et porteur de mauvaises nouvelles. La dynamique de compétition entre R&D et qualité transforme ce qui devrait être une collaboration en confrontation.

Élément absent ou flou	Ce qui devient flou	Conséquences possibles	Symptômes observables

Qualité intégrée au développement	Responsabilité qualité dans le flux de développement quotidien.	La qualité est perçue comme une étape finale plutôt qu'une pratique continue; les développeurs peuvent se déresponsabiliser.	Tests repoussés en fin de cycle, « c'est la job de QA », défauts découverts trop tard, rework systématique.
Qualité intégrée aux opérations	Lien entre qualité de livraison, stabilité opérationnelle et retour terrain.	Les signaux opérationnels ne remontent pas vers les pratiques de développement; les mêmes problèmes se répètent.	Incidents récurrents, correctifs réactifs, absence de boucle de rétroaction entre production et développement.
Qualité intégrée au portfolio	Prise en compte de la qualité dans les décisions d'investissement, de priorisation et de capacité.	La qualité est sacrifiée au profit de la vélocité ou du scope; la dette qualité s'accumule sans visibilité.	Arbitrages qui ignorent la dette technique, roadmap sans critères qualité, surprises en fin de release.
Qualité comme catalyseur, et non comme contrôle	Rôle de la qualité : assistance, facilitation et accélération plutôt que contrôle et blocage.	La qualité est perçue comme un ennemi, un fauteur de trouble ou un diseur de mauvaises nouvelles.	Résistance aux pratiques qualité, contournement des processus, équipes qui cachent les problèmes plutôt que de les exposer.
Climat de confiance entre R&D et qualité	Relation de collaboration, d'entraide et de respect mutuel entre développement et qualité.	Le R&D se positionne en mode compétition et reçoit les recommandations qualité comme une remise en question de sa compétence; la qualité, de son côté, cherche parfois la	Sentiment d'être jugé, complexe d'infériorité ou de supériorité, rejet systématique des recommandations, frustration généralisée, discussions qui tournent en

		recette parfaite et impose des standards perçus comme déconnectés.	confrontation plutôt qu'en résolution.
Posture d'humilité et d'ouverture	Capacité de chaque partie à reconnaître ses limites, à écouter et à ajuster sans se sentir menacée.	Le R&D rejette les recommandations parce qu'il pense déjà maîtriser la situation; la qualité s'entête parfois sur des pratiques théoriques sans assez tenir compte du contexte réel.	Mode défensif permanent, « nous contre eux », absence de remise en question constructive, perte de focus sur les vrais enjeux au profit de l'ego.

Effet combiné : la qualité seule, sans contexte, n'existe tout simplement pas. Elle ne doit pas être séparée du développement, des opérations ni du portfolio. Lorsqu'elle est isolée, elle devient une fonction externe qui juge, ralentit ou annonce les mauvaises nouvelles. Les équipes se déresponsabilisent, les recommandations sont rejetées et chacun protège son territoire au lieu de collaborer sur les risques réels.

Recommandation : cesser de traiter la qualité comme une fonction séparée ou un département autonome. La qualité doit être intégrée dans les pratiques quotidiennes, présente dans chaque phase du SDLC, rattachée à un contexte concret et portée comme un catalyseur du centre d'excellence. Elle doit aider les équipes à prendre de meilleures décisions, pas imposer une recette parfaite.

La qualité doit être :

- **Intégrée** — présente du portfolio à la production, pas seulement en fin de cycle.
- **Contextuelle** — toujours rattachée à un produit, une livraison, une opération, une décision ou un risque concret.
- **Catalyseur** — un accélérateur qui rend les équipes meilleures, pas un contrôleur qui ralentit ou bloque.
- **Facilitatrice** — une assistance évidente partout, qui aide à résoudre les enjeux plutôt qu'à chercher un coupable.
- **Partagée** — la responsabilité qualité appartient à tous, pas seulement à une équipe dédiée.

- **Bienveillante** — la qualité ne juge pas, elle accompagne; le R&D ne doit pas se sentir en compétition avec la qualité, et la qualité ne doit pas se sentir en mission de correction permanente.

Point de vigilance — dynamique relationnelle : lorsque le R&D est en mode « nous sommes les meilleurs » et que la qualité est en mode « voici la recette parfaite », les deux parties se retrouvent en posture défensive. Le R&D perçoit chaque recommandation comme une attaque à sa compétence; la qualité perçoit chaque résistance comme un manque de rigueur. Ce cercle vicieux doit être brisé par l'humilité des deux côtés, le focus sur les vrais enjeux, la co-construction des recommandations et la reconnaissance mutuelle des expertises.

Point de vigilance : il faut éviter à tout prix que la qualité soit perçue comme un ennemi, un fauteur de trouble ou un diseur de mauvaises nouvelles. Si cette perception existe, c'est le symptôme d'un positionnement organisationnel défaillant. La solution n'est pas seulement de mieux communiquer la qualité, mais de la repositionner comme une composante naturelle et indissociable du travail quotidien.

Ajustement de la responsabilité produit complète (SENSE)

La voix client, produit et livraison n'est pas suffisamment explicitée.

> ⚠ **Conséquences si les rôles Produit et Livraison ne sont pas explicitement assignés**
>
> Si les rôles de **Responsable produit (Product Manager — PM)**, **Propriétaire du produit (Product Owner — PO)** et **Chargé de livraison (Delivery Manager — CdL)** ne sont pas clairement définis, les décisions clés deviennent implicites, distribuées ou prises au cas par cas. Le risque principal est de confondre **prioriser**, **spécifier**, **implémenter** et **livrer**.

Élément absent ou flou	Ce qui devient flou	Conséquences possibles	Symptômes observables
Responsable produit (Product Manager — PM)	Vision produit, roadmap, arbitrage marché/client, stratégie de maintenance, fin de vie et transition produit.	Priorisation réactive, roadmap implicite, arbitrages incohérents entre dette technique, demandes clients, RT-LAB et HYPER-LAB.	Backlog peu orienté valeur, clients influents qui dictent les priorités, difficulté à expliquer pourquoi une demande est acceptée, refusée ou reportée.

Propriétaire du produit (Product Owner — PO)	Backlog tactique, découpage des besoins, stories, spécifications IF… THEN…, Definition of Ready et priorisation d'équipe.	Stories insuffisamment clarifiées, spécifications incomplètes, spécifications d'acceptation tardives, rework pendant l'implémentation.	Allers-retours fréquents avec les équipes, stories trop grosses, débats tardifs sur ce qui était attendu, difficultés à savoir si un item est prêt à développer.
Chargé de livraison (Delivery Manager — CdL)	Respect du SDLC, critères de sortie, dépendances, métriques, signaux de risque, préparation Go/No-Go et transfert vers les opérations/support.	La livraison devient une accumulation de tâches plutôt qu'un flux maîtrisé; les risques sont découverts trop tard; les silos réapparaissent entre produit, développement, validation, opérations et support.	Surprises en fin de release, checklists incomplètes, support mal préparé, décisions basées sur l'intuition plutôt que sur les métriques et les signaux objectifs.

Effet combiné : sans PM, PO et CdL clairement assignés, les mêmes personnes finissent par prendre plusieurs décisions implicitement, sans mandat clair. Cela fragilise la traçabilité, la qualité, la collaboration transverse et la capacité à livrer de façon prévisible.

Recommandation : ces responsabilités doivent exister même si elles ne correspondent pas à des postes à temps plein. Pour un produit en maintenance comme RT-LAB, les responsabilités PM et PO peuvent être combinées ou partagées avec le produit successeur, par exemple HYPER-LAB, mais elles ne devraient pas être absentes. L'absence de nouvelles fonctionnalités ne signifie pas l'absence de décisions produit.

Note : les descriptions de PM, PO et CdL dans cette section sont des responsabilités recommandées pour couvrir la voix produit et la livraison dans ce contexte. Pour les définitions générales des rôles et la répartition des responsabilités, se référer au Lexique / Terminologies et à la matrice RACI.

Point de vigilance : un produit en maintenance requiert quand même une responsabilité produit explicite, notamment pour arbitrer la dette, les correctifs, les incidents, les dépendances, la fin

de vie et les transitions vers les produits successeurs.

Ajustement de la voix du client (Customer Voice)

Les signaux clients sont dispersés et ne sont pas encore structurés comme un mécanisme de décision produit.

> ⚠ **Conséquences si la voix du client n'est pas structurée**
>
> Sans mécanisme explicite de voix du client, les décisions produit reposent surtout sur les perceptions internes, les urgences ponctuelles ou les clients les plus bruyants. Le risque est de perdre la traçabilité entre les irritants réels, la valeur attendue, la priorisation produit et les choix de livraison.

Élément absent ou flou	Ce qui devient flou	Conséquences possibles	Symptômes observables
Canal de rétroaction intégré au produit	Recueil continu des irritants, suggestions et besoins directement dans l'application.	Les retours clients arrivent par des canaux dispersés et non comparables.	Demandes perdues dans les courriels, Slack ou rencontres; difficulté à regrouper les tendances.
Sondages et mesures de satisfaction	Niveau de satisfaction, perception de qualité, irritants récurrents et priorités perçues.	Les décisions sont basées sur quelques anecdotes plutôt que sur un signal représentatif.	Débats internes sur ce que "les clients veulent vraiment"; manque de données comparables dans le temps.
Suivi incident / version	Lien entre incidents, versions, correctifs, clients affectés et impact opérationnel.	Les correctifs sont priorisés sans compréhension claire de l'impact client réel.	Retours récurrents sur les mêmes problèmes, difficulté à expliquer pourquoi un correctif est prioritaire.
Télémétrie et traçabilité acceptées par l'utilisateur	Usage réel, fréquence des erreurs, adoption des fonctionnalités et parcours problématiques.	Le produit est amélioré selon ce que l'on croit être utilisé, plutôt que selon l'usage réel.	Fonctionnalités peu utilisées qui continuent d'être maintenues; irritants

			fréquents non visibles par l'équipe.
Envoi automatisé d'erreurs et crashs	Fréquence, contexte, sévérité et reproductibilité des problèmes techniques côté client.	Les défauts critiques sont découverts tard ou seulement lorsqu'un client les rapporte manuellement.	Temps long pour reproduire les problèmes, correctifs réactifs, manque de priorisation basée sur la sévérité réelle.

Effet combiné : sans voix du client structurée, l'organisation priorise selon des signaux incomplets. Les incidents, les irritants, l'usage réel et la satisfaction ne sont pas reliés dans un même système de décision, ce qui affaiblit la roadmap, les arbitrages qualité et la préparation des livraisons.

Recommandation : mettre en place un mécanisme de voix du client combinant rétroaction intégrée, sondages, suivi incident/version, télémétrie consentie et envoi automatisé d'erreurs. Ces signaux doivent alimenter le Responsable produit (Product Manager — PM), le Propriétaire du produit (Product Owner — PO), le Chargé de livraison (Delivery Manager — CdL) et le comité Portfolio afin d'appuyer la priorisation, les arbitrages et les décisions Go/No-Go.

Mécanismes recommandés

- UserVoice ou un mécanisme équivalent de rétroaction directement dans l'application.
- Sondages périodiques pour mesurer la satisfaction, la perception de qualité et les irritants récurrents.
- Suivi incident / version pour relier les problèmes aux versions livrées, aux clients affectés et aux correctifs.
- Mécanisme de télémétrie et de traçabilité embarquée, accepté par l'utilisateur, pour soutenir l'amélioration continue de l'application.
- Mécanisme d'envoi automatisé d'erreurs et de crashs afin de réduire le délai de détection et de reproduction.

Point de vigilance : la voix du client ne doit pas être confondue avec la voix du client le plus bruyant. Les données doivent être consolidées, qualifiées et mises en contexte pour éviter que la roadmap soit dictée uniquement par l'urgence ou l'anecdote.

Ajustement du contrat qualité

Aucune entente qualité commune n'est formalisée à ce jour.

⚠ **Conséquences si le contrat qualité n'est pas formalisé**

Sans contrat qualité explicite, chaque équipe interprète différemment ce qui est acceptable pour spécifier, développer, tester, livrer et supporter. La qualité devient une négociation de fin de cycle plutôt qu'un engagement partagé dès le départ.

Élément absent ou flou	Ce qui devient flou	Conséquences possibles	Symptômes observables
Critères d'entrée et de sortie par phase	Quand une phase peut commencer, se terminer et passer à la suivante.	Des travaux passent en développement ou en livraison sans être réellement prêts.	Rework, blocages tardifs, débats sur "qui aurait dû valider quoi".
Definition of Ready et Definition of Done	Ce qui rend un item prêt à développer et ce qui le rend terminé.	Les stories avancent avec des spécifications incomplètes ou une qualité variable.	Stories rouvertes, spécifications d'acceptation ajoutées tard, validations oubliées ou incomplètes.
Critères de dérogation	Quand une exception qualité est acceptable, qui l'autorise et comment elle est suivie.	Les exceptions deviennent implicites, répétées ou non assumées.	Décisions orales, risques acceptés sans trace, dette qualité accumulée.
Preuves et métriques qualité	Quels signaux sont nécessaires pour démontrer la qualité et la préparation à livrer.	Les décisions Go/No-Go reposent sur l'intuition plutôt que sur des preuves.	Tableaux de bord incomplets, validations non liées aux spécifications, couverture difficile à expliquer.

Effet combiné : sans contrat qualité commun, les attentes qualité se déplacent d'une équipe à l'autre et d'une livraison à l'autre. Les spécifications, preuves et exceptions deviennent négociées trop tard, les dérogations s'accumulent et les décisions de livraison reposent davantage sur la pression du moment que sur des preuves partagées.

Recommandation : formaliser un contrat qualité commun décrivant les critères d'entrée/sortie, les responsabilités, les preuves attendues, les dérogations et les métriques minimales. Le contrat doit être utilisé comme outil de collaboration entre produit, développement, validation, opérations, support et livraison — pas comme une checklist administrative de fin de cycle.

Point de vigilance : le contrat qualité doit rester vivant, simple et utile. S'il devient trop lourd, il sera contourné; s'il est trop vague, il ne changera pas les comportements.

Ajustement de la centralisation de la documentation produit et des spécifications

La documentation produit commune, synchronisée, dynamique, modélisée et testable n'est pas suffisamment centralisée.

> ⚠ **Conséquences si la documentation produit et les spécifications ne sont pas centralisées**
>
> Sans documentation produit commune, synchronisée, dynamique et testable, chaque fonction travaille avec sa propre interprétation du besoin. Le risque principal est de créer des silos entre produit, développement, validation, opérations et support, puis de découvrir les écarts seulement à la fin.

Élément absent ou flou	Ce qui devient flou	Conséquences possibles	Symptômes observables
Source unique de vérité produit	Où se trouvent les spécifications, décisions, modèles, hypothèses et preuves de validation à jour.	Les équipes travaillent avec des versions différentes de la réalité produit.	Décisions répétées, documents contradictoires, questions récurrentes déjà tranchées ailleurs.
Modélisation commune	Comportements attendus, règles d'affaires, spécifications IF... THEN..., états, interfaces et dépendances.	Les impacts sont découverts trop tard et les validations ne couvrent pas les bons comportements.	Défauts d'intégration, incompréhensions entre équipes, modèles informels dans la tête de quelques experts.

Spécifications testables	Lien entre besoin, spécifications IF... THEN..., validations, preuves et définition de terminé.	La validation devient interprétative plutôt que vérifiable.	Tests créés tard, couverture difficile à démontrer, débats sur ce qui devait être validé.
Participation de tous les joueurs concernés	Qui doit contribuer aux ateliers de définition, de modélisation et de validation.	Les décisions sont prises en silo et doivent être revalidées après coup.	Allers-retours fréquents, dépendances oubliées, support ou opérations impliqués trop tard.

Effet combiné : lorsque la documentation produit et les spécifications ne sont pas centralisées, la compréhension du produit devient fragmentée. Les décisions, modèles, spécifications et preuves de validation se dispersent entre outils, personnes et équipes, ce qui augmente le rework et réduit la capacité de livrer avec confiance.

Recommandation : établir une documentation produit commune dans Confluence, reliée aux modèles, aux stories, aux validations Xray et aux décisions. Les ateliers de modélisation et de spécification doivent inclure les parties prenantes concernées dès le départ afin que le besoin soit compris, testable et livrable sans revalidation tardive.

Point de vigilance : la documentation produit ne doit pas devenir un entrepôt statique. Elle doit rester connectée à la livraison, aux validations, aux décisions et aux changements réels du produit.

Ajustement du modèle de versionnement et du découpage des livraisons

Le modèle de versionnement, l'alignement des standards et le découpage des paquets de livraison ne sont pas suffisamment uniformisés.

> ⚠ **Conséquences si le modèle de versionnement et le découpage des livraisons ne sont pas uniformisés**
>
> Sans règles communes de versionnement, de découpage Jira et de paquets de livraison, chaque équipe peut interpréter différemment ce qui constitue une livraison majeure, mineure, corrective ou technique. Le risque principal est de perdre l'alignement entre valeur produit, traçabilité technique, effort de validation et communication client.

Élément absent ou flou	Ce qui devient flou	Conséquences possibles	Symptômes observables
Niveaux de version	Distinction entre version majeure, mineure, patch et build.	Les équipes utilisent les mêmes mots pour désigner des réalités différentes.	Confusion entre communication client, documentation, packaging, branches, commits et pipelines.
Unité principale de livraison	Niveau Jira à utiliser pour regrouper la valeur livrable.	Les livraisons deviennent trop grosses, trop petites ou mal alignées avec la valeur produit.	EPIC interminables, FEATURE trop vagues, stories regroupées tardivement, difficulté à expliquer ce qui est réellement livré.
Granularité des paquets	Nombre d'éléments à inclure dans une livraison donnée.	Le regroupement excessif augmente le risque, la complexité, le coût de validation et le stress opérationnel.	Campagnes de tests massives, anomalies difficiles à isoler, cycles imprévisibles, défauts découverts tard.
Traçabilité Jira / version	Lien entre EPIC, FEATURE, STORY/BUG, commit, build et version livrée.	La preuve de ce qui est livré, testé et supporté devient difficile à reconstituer.	Release notes incomplètes, audits difficiles, décisions Go/No-Go appuyées sur une vue partielle.

Effet combiné : lorsque les niveaux de version, les paquets Jira et les unités de valeur ne sont pas alignés, la livraison devient difficile à expliquer, à tester et à supporter. Les équipes regroupent trop d'éléments, la validation devient lourde et la traçabilité entre décision, code, test, build et version livrée s'affaiblit.

Recommandation : établir une règle commune reliant chaque niveau de découpage Jira à un niveau de version. La FEATURE devrait être l'unité principale de livraison produit, en conservant une relation aussi proche que possible de 1:1 entre paquet Jira et niveau de version afin de réduire la complexité et de maîtriser la validation.

Niveaux de version recommandés

Usage	Format	Intention
Interne — traçabilité complète	[majeur].[mineur].[patch].[build]	Aligné avec les branches, les pipelines CI/CD, les commits et la traçabilité technique.
Documentation et packaging	[majeur].[mineur].[patch]	Format lisible et précis pour la documentation, les notes de version et le packaging.
Client, utilisateur final et vente	[majeur].[mineur]	Format simplifié, axé sur la valeur fonctionnelle livrée.

Alignement entre découpage Jira et versionnement

Niveau Jira	Objectif	Niveau de version	Exemple
EPIC	Valeur affaire (business)	Livraison majeure	**2026**.x.x.x
FEATURE	Valeur produit et unité principale de livraison recommandée	Livraison mineure	x.**1**.0.x
STORY / BUG	Montage, réalisation ou correction	Patch	x.1.**1**.x
COMMIT / BUILD	Traçabilité technique	Build	x.x.x.**3f5a9c8**

Clarification importante — définition de FEATURE

Dans ce contexte, **FEATURE** ne signifie pas une fonctionnalité isolée, un fichier Cucumber ou une petite capacité livrée indépendamment. Selon la définition de référence du Lexique / Terminologies, et pour cette gouvernance de versionnement, **FEATURE = niveau de livraison produit**.

Une FEATURE représente un regroupement cohérent de valeur produit, un incrément livrable au niveau produit et un niveau de version mineur. On conserve le terme FEATURE, mais sa signification locale doit être explicite afin d'éviter la confusion avec l'usage courant du mot "feature" dans l'industrie.

Règles de granularité recommandées

- Éviter autant que possible plusieurs STORIES/BUGS dans un même PATCH.
- Éviter plusieurs modifications non reliées dans un même COMMIT.
- Éviter plusieurs FEATURES dans une même version mineure.
- Éviter plusieurs EPICS dans une même version majeure.
- Appliquer une règle simple : relation 1:1 entre le niveau Jira et le niveau de version correspondant, lorsque le contexte le permet.

Pourquoi : chaque regroupement augmente le risque, la complexité, le coût de validation et le stress opérationnel. Le risque n'augmente pas linéairement; il peut augmenter de manière exponentielle lorsque plusieurs changements sont combinés dans une même livraison.

Piège classique — l'illusion de la livraison rapide

Certaines organisations croient aller plus vite en regroupant les livraisons, en mobilisant les équipes moins souvent ou en considérant que les petits livrables coûtent trop cher en coordination. Dans la majorité des cas, ce raisonnement produit l'effet inverse : les tests deviennent plus lourds, les anomalies sont plus difficiles à isoler, les corrections coûtent plus cher et les délais explosent.

Le mythe selon lequel une EPIC ou une FEATURE devrait nécessairement durer plusieurs mois ou plusieurs années doit aussi être questionné. Une livraison majeure ou mineure peut être courte si elle représente une valeur clairement découpée et maîtrisée.

Bénéfices attendus

- Cohérence entre produit, technique, documentation et livraison.
- Traçabilité complète entre besoin, Jira, test, commit, build et version.
- Découpage plus naturel et plus facile à expliquer.
- Réduction de la confusion utilisateur et de la confusion interne.
- Tests plus simples, plus fiables et moins massifs.
- Réduction des échecs, du rework et du stress global.

Point de vigilance : la terminologie doit rester uniforme. Certaines équipes utilisent "majeur / mineur" pour désigner en réalité "mineur / patch". Les définitions doivent être clarifiées avant de comparer les métriques, les versions ou les engagements de livraison.

Usage structuré de l'IA dans le SDLC

L'IA est déjà utilisée de façon ponctuelle pour la lecture de code, l'analyse de tickets Jira, le débogage, la description des tâches et la génération de cas de validation. Cet usage devient un

levier réel seulement s'il est structuré, traçable et intégré au SDLC.

- Utiliser l'IA pour soutenir l'analyse de code, la revue des tickets Jira, la compréhension du contexte des bugs et la génération de cas de validation.
- Résumer et valider le contexte des bugs et fonctionnalités avec l'aide de l'IA, tout en conservant une validation humaine explicite.
- Encadrer l'usage avancé de l'IA par des règles simples : sources fiables, contexte clair, validation humaine, traçabilité des décisions et protection des informations sensibles.
- Rappeler que l'IA ne peut être efficace que si les spécifications, modèles et sources de vérité sont fiables, à jour et accessibles.

Élément absent ou flou	Ce qui devient flou	Conséquences possibles	Symptômes observables
Cadre d'utilisation de l'IA	Quand utiliser l'IA, pour quelles tâches, avec quelles sources et quelles limites.	Usage ad hoc et variable entre équipes; résultats ni comparables ni reproductibles.	Pratiques disparates, qualité variable, aucune capitalisation collective.
Validation humaine explicite	Distinction entre suggestion IA et décision validée par un humain.	Les sorties IA sont acceptées sans vérification; des erreurs subtiles passent en production ou en documentation.	Spécifications ou cas de test plausibles mais incorrects, confiance excessive.
Traçabilité de l'assistance IA	Savoir quels artefacts ont été produits ou enrichis avec l'aide de l'IA.	Impossible de distinguer ce qui a été validé par un expert de ce qui a été généré automatiquement.	Doutes sur la fiabilité des artefacts, risque de régression des compétences.
Qualité des sources alimentant l'IA	Fiabilité des spécifications, modèles et données utilisés comme contexte.	L'IA produit des résultats cohérents en apparence mais basés sur des sources obsolètes ou contradictoires.	« Garbage in, garbage out » — l'IA amplifie les lacunes existantes.

Recommandation : intégrer l'IA comme assistance au raisonnement, à la synthèse et à la préparation des validations, sans la traiter comme une source de vérité autonome. L'IA doit accélérer la compréhension et la modélisation, mais les décisions produit, qualité et livraison doivent demeurer validées par les responsables humains.

Automatisation des validations et couverture ciblée

L'automatisation des validations doit être orientée vers les états importants des sous-systèmes plutôt que vers une explosion exhaustive de toutes les combinaisons possibles. L'objectif est de réduire l'ambiguïté, de rendre les preuves répétables et de préparer le passage à l'échelle sans créer une suite de validation ingérable.

- Explorer l'automatisation et le contrôle à distance du matériel pour les validations, en tenant compte des accès, jetons, Vault, autorisations, contraintes techniques et exigences de sécurité.
- Définir une couverture de validation ciblée sur les états importants des sous-systèmes et les risques réels plutôt que sur toutes les combinaisons théoriques.
- Identifier quatre cas de validation minimaux pour couvrir les états critiques des sous-systèmes et servir de base de modélisation.
- Documenter les cas de validation comme des spécifications produit afin de faciliter la modélisation, la traçabilité Xray et l'automatisation.

Recommandation : relier chaque validation automatisable à une spécification produit claire, observable et validable. L'automatisation doit confirmer les comportements attendus et soutenir les décisions de livraison, pas seulement augmenter le nombre de validations exécutées.

Architecture de modélisation et d'automatisation — AltWalker + pytest + Robot Framework + Xray

La modélisation comportementale et l'automatisation des validations doivent être soutenues par une architecture intégrée qui couvre les trois niveaux de validation : **code** avec pytest, **utilisateur et système** avec Robot Framework, et **comportement produit modélisé** avec AltWalker / GraphWalker. Cette architecture doit être reliée à **Xray** pour la traçabilité et la couverture, et intégrer le **design système embarqué** comme source de modélisation.

Pourquoi cette architecture

- **AltWalker**, implémentation Python de GraphWalker, permet de modéliser les comportements produit sous forme de graphes — états, transitions, règles d'affaires — et de générer dynamiquement des chemins de validation par Model-Based Testing.

- **pytest** couvre les validations au niveau du code : tests unitaires, intégration technique, TDD et vérifications applicatives.
- **Robot Framework** couvre les validations utilisateur et système : acceptance, bout en bout, interfaces, scénarios fonctionnels et intégration matérielle.
- **Xray** relie les résultats d'exécution aux spécifications IF… THEN…, test sets, plans de validation et couverture produit.
- **Le design système embarqué** — diagrammes d'architecture, interfaces matériel/logiciel, flux de données — doit alimenter les modèles AltWalker et les spécifications Xray pour garantir que la modélisation reflète la réalité du produit.

Élément absent ou flou	Ce qui devient flou	Conséquences possibles	Symptômes observables
Architecture de modélisation unifiée	Quel outil modélise quoi, à quel niveau et avec quelle traçabilité.	Chaque équipe choisit sa propre approche; les modèles ne sont pas reliés aux validations ni à la couverture produit.	Modèles isolés, non exécutables, non liés à Xray, duplication entre documentation et tests.
Lien entre modèles et spécifications Xray	Comment les comportements modélisés deviennent des validations traçables.	Les modèles restent des artefacts de documentation; les validations ne couvrent pas les comportements modélisés.	Modèles et tests qui évoluent indépendamment, couverture déclarative mais non vérifiable.
Design système embarqué comme source de modélisation	Comment l'architecture matériel/logiciel, les interfaces et les flux alimentent la modélisation comportementale.	Les modèles décrivent des comportements abstraits sans ancrage suffisant dans la réalité technique du produit.	Modèles plausibles mais déconnectés du matériel, validations qui passent sur simulateur mais échouent sur cible réelle.
Intégration pytest + Robot Framework + AltWalker	Comment les trois niveaux de validation — code, système,	Duplication de validations entre les outils, effort de	Tests redondants, suites fragiles, confusion sur quel

	comportement — se complètent sans se dupliquer.	maintenance multiplié, faux sentiment de couverture.	outil valide quel niveau.

Recommandation : définir une architecture de modélisation et d'automatisation intégrant AltWalker pour la modélisation comportementale et le MBT, pytest pour le code et l'intégration technique, Robot Framework pour les validations utilisateur et système, et Xray pour la traçabilité et la couverture. Le design système embarqué doit être une source de modélisation afin de garantir l'ancrage dans la réalité produit.

Point de vigilance : l'investissement initial en modélisation est modéré mais réel. La valeur se manifeste à moyen terme par la réduction des tests dupliqués, l'amélioration de la couverture, la documentation produit vivante et la capacité de générer dynamiquement des validations pertinentes.

Ajustement de la gouvernance des rôles et du pouvoir décisionnel

Les rôles à pouvoir décisionnel peuvent devenir une source de collision lorsqu'ils ne sont pas appuyés par des responsabilités explicites, un RACI clair et des contrats ou conventions partagés. Le risque n'est pas seulement de créer des tensions entre personnes : il est de concevoir une structure où plusieurs rôles peuvent légitimement croire qu'ils décident de la même chose.

> ⚠ **Conséquences si les rôles décisionnels ne sont pas différenciés**
>
> Quand deux rôles peuvent dire « c'est moi qui décide » sur la même responsabilité, le conflit est structurel et inévitable. Le problème n'est alors pas une question de personnalité, mais une ambiguïté de conception organisationnelle.

Configuration	Risque structurel	Conséquence probable
Directeur architecte sous le VP R&D, Directeur qualité à côté du R&D	L'architecture a un accès direct au pouvoir décisionnel R&D; la qualité peut être perçue comme externe, sans levier opérationnel direct.	La qualité devient un frein perçu plutôt qu'un catalyseur. Les recommandations qualité peuvent être rejetées, contournées ou traitées comme des contraintes externes.

Directeur architecte et Directeur qualité au même niveau sous le VP R&D	Les deux rôles ont un levier similaire, mais leurs mandats peuvent se chevaucher sur les standards, les revues, la conformité, les arbitrages techniques et les critères de livraison.	Compétition, doublons, escalades fréquentes et arbitrages constants du VP R&D, qui devient alors un goulot d'étranglement.
Un seul rôle cumule architecture et qualité	La collision est réduite, mais le rôle devient surchargé et risque de développer des angles morts.	Selon les urgences, la qualité ou l'architecture finit par être sacrifiée. Les décisions peuvent devenir moins équilibrées.

Recommandation : avant de créer, maintenir ou renforcer un rôle à pouvoir décisionnel, répondre explicitement aux questions suivantes :

1. Quelles tâches spécifiques ce rôle réalise-t-il que personne d'autre ne fait ?
2. Quelle décision finale ce rôle approuve-t-il — le A dans le RACI — sans ambiguïté ?
3. Qui est consulté avant la décision, et qui est seulement informé après la décision ?
4. Si ce rôle disparaît, qui absorbe ses responsabilités, et est-ce déjà explicite ?
5. Est-ce que deux rôles répondent la même chose aux questions précédentes ? Si oui, il faut fusionner, différencier ou redéfinir les responsabilités.

Principe fondamental — tâches et responsabilités d'abord, rôles ensuite

La qualité, l'agilité, l'architecture, la modélisation, l'automatisation et l'observabilité sont toutes des pratiques de la réalisation de produit logiciel. Aucune de ces pratiques ne peut exister sans contexte : elles prennent sens dans le produit, le flux SDLC, les contraintes techniques, les utilisateurs, les opérations et les objectifs de livraison.

Il ne faut donc pas créer des rôles qui font la même chose sous des noms différents. La différence se fait en distinguant précisément les tâches, les responsabilités, les décisions et les interfaces entre rôles.

Anti-pattern	Pourquoi c'est un problème
Créer un rôle qualité qui décide seul des standards qualité	La qualité est une propriété du produit, pas seulement d'un département. Les standards

	doivent être co-construits avec ceux qui les appliquent et ceux qui en assument les risques.
Créer un rôle architecture qui décide seul de l'architecture	L'architecture émerge aussi des contraintes de livraison, de validation, d'opérations, de maintenance, de performance et d'évolution produit.
Créer un rôle agile permanent qui impose l'agilité	L'agilité est une pratique d'équipe et de flux. Un rôle catalyseur qui ne transfère pas la compétence risque de devenir un rôle compensatoire.
Multiplier les rôles décisionnels sans clarifier les tâches	Chaque nouveau rôle décisionnel crée de nouvelles interfaces, de nouveaux arbitrages et de nouveaux risques de collision.

La règle :

1. Lister les tâches et responsabilités dans la colonne de gauche du RACI.
2. Distribuer R, A, C et I en gardant un seul A par ligne.
3. Nommer les rôles seulement après, en fonction de ce qui doit être fait, pas seulement en fonction de l'organigramme.
4. Vérifier qu'aucun rôle ne fait la même chose qu'un autre. Si oui, fusionner ou différencier.
5. Distinguer les rôles permanents, les rôles catalyseurs et les rôles compensatoires.

Principe de maturité : il ne faut pas chercher à remplir tous les rôles. Il faut d'abord comprendre les tâches et responsabilités, puis distribuer les rôles selon la capacité, la maturité et le besoin réel de l'organisation. Bien différencier les tâches et responsabilités est ce qui fait la différence entre une organisation qui collabore et une organisation qui compense.

Recommandation — privilégier les rôles catalyseurs-cadres au décisionnel

Principe : les rôles catalyseurs-cadres plutôt que les rôles décisionnels. Un rôle catalyseur-cadre accompagne, structure, outille et rend visible — mais ne décide pas à la place des équipes. Le pouvoir décisionnel doit être appuyé par des **contrats, conventions, ententes, DoR/DoD, critères de sortie et règles de gouvernance** explicites, plutôt que porté uniquement par une personne ou un titre.

Cette approche réduit fortement les conflits, car elle déplace l'arbitrage du rapport de force entre rôles vers un cadre commun, visible et vérifiable. Le rôle catalyseur-cadre aide les équipes

à appliquer le cadre, à l'améliorer et à détecter les écarts, sans devenir un passage obligé pour chaque décision.

Aspect	Rôle décisionnel	Rôle catalyseur-cadre
Pouvoir	Décide pour les autres ou devient le point d'arbitrage principal.	Rend les autres capables de décider dans un cadre clair.
Source d'autorité	L'organigramme, le titre, l'ancienneté ou le rapport de force.	Le contrat qualité, les conventions, le RACI, les critères de sortie, la DoR/DoD et les ententes explicites.
En cas de conflit	Arbitrage personnel, variable selon la personne, le moment ou la pression.	Arbitrage par le cadre : prévisible, impersonnel, documenté et vérifiable.
Scalabilité	Risque de goulot d'étranglement : les décisions remontent toujours aux mêmes personnes.	Décisions distribuées : les équipes appliquent le cadre et escaladent seulement les exceptions.
Effet sur la maturité	Peut maintenir la dépendance à une personne experte ou à une direction.	Transfère la compétence, clarifie les responsabilités et accélère l'autonomie des équipes.
Quand la personne part	Le pouvoir décisionnel disparaît ou devient flou.	Le cadre reste; les équipes continuent avec des règles connues.

Effets recherchés :

1. **Réduire les conflits** — quand le contrat, la convention ou l'entente tranche, il y a moins de collision entre personnes ou directions.
2. **Rendre visibles les rôles non nécessaires** — si un rôle n'a plus rien à décider une fois le cadre clarifié, il est peut-être compensatoire ou doit devenir catalyseur.
3. **Accélérer la maturité** — les équipes internalisent les standards au lieu de dépendre d'un décideur central.

4. **Limiter les décisions exceptionnelles** — le décisionnel reste nécessaire, mais il se concentre sur les vraies exceptions, les arbitrages stratégiques et les risques assumés.

Au lieu de…	Privilégier…
Le Directeur qualité décide seul si une livraison est conforme.	Le **contrat qualité** définit les critères; l'équipe vérifie; les métriques rendent l'état visible; l'autorité finale assume seulement les dérogations ou exceptions.
Le Directeur architecte approuve chaque décision technique.	Les **conventions d'architecture et de modélisation** définissent les patterns; les équipes appliquent; les revues détectent les écarts; les exceptions sont documentées.
Le VP R&D arbitre fréquemment entre architecture, qualité, livraison et opérations.	Le **RACI** clarifie les responsabilités; chaque tâche a un seul A; les pratiques transversales se consultent sans se chevaucher.
Un coach qualité ou agile devient le gardien permanent de la conformité.	Le coach agit comme **catalyseur-cadre** : il accompagne, forme, challenge et transfère la compétence jusqu'à ce que l'équipe soit autonome.

⚠ **Test de nécessité d'un rôle**

Si on remplace la personne par un contrat, une convention ou une entente explicite et que les équipes fonctionnent toujours, le rôle est probablement compensatoire, trop décisionnel ou devrait être repositionné comme catalyseur-cadre. À l'inverse, si le cadre révèle une vraie responsabilité non couverte, le RACI doit l'assigner explicitement.

Ajustement de la posture qualité dans les équipes de développement

Il ne devrait pas y avoir de rôle « QA » monolithique traditionnel dans une équipe de développement moderne. La qualité est une responsabilité distribuée entre les personnes qui définissent le produit, conçoivent l'architecture, écrivent le code, valident les comportements, livrent, opèrent et soutiennent les utilisateurs.

⚠ **Conséquences si la qualité reste portée par un rôle QA fourre-tout**

Lorsque la qualité est confiée à un rôle unique en fin de chaîne, les équipes se déresponsabilisent, les validations arrivent trop tard et les problèmes de spécification, de

conception ou d'implémentation sont découverts au moment le plus coûteux.

Principe directeur : la mauvaise question est *« est-ce au QA de… ? »*. La bonne question est *« qui est le mieux placé pour… ? »*. Dans une équipe dev moderne, il n'y a pas un QA qui fait tout — il y a des responsabilités qualité distribuées.

- **Dev / Arch** → responsable de la qualité de son code : tests unitaires, TDD lorsque pertinent, revue de code, testabilité et Definition of Done.
- **Ingénieur ou spécialiste en validation** → responsable de la stratégie de validation, des tests système, intégration, acceptance, traçabilité et Xray.
- **Chargé de livraison** → responsable de la conformité processus, des critères de sortie, du dossier Go/No-Go et des signaux de livraison.
- **Responsable du développement** → imputable de la qualité technique globale dans le périmètre développement.
- **Coach qualité** → catalyseur des pratiques qualité; accompagne, challenge et transfère les compétences sans faire la qualité à la place de l'équipe.

Questions de réflexion — réponses recommandées

Question	Réponse recommandée	Pourquoi
Qui rédige les validations basées sur les spécifications IF… THEN… ?	**Ingénieur ou spécialiste en validation** pour les tests système, intégration et acceptance. **Dev / Arch** pour les tests unitaires et les tests techniques.	Les validations découlent des spécifications IF… THEN… — celui qui comprend le besoin et le risque est le mieux placé. Les tests unitaires sont indissociables du code.
Qui exécute les tests ?	**Dev / Arch** exécute ses tests unitaires avant merge. **Ingénieur ou spécialiste en validation** exécute les tests système, intégration et acceptance. **L'automatisation** exécute en continu.	L'exécution manuelle devrait être l'exception. L'objectif est l'automatisation maximale. Le dev est responsable de valider son propre travail avant de le soumettre.
Qui automatise les tests ?	**Dev / Arch** automatise les tests unitaires et d'intégration technique. **Ingénieur ou spécialiste en validation**	Celui qui écrit ou conçoit le test est souvent le mieux placé pour l'automatiser. Le développement ne devrait pas

	automatise les tests système, acceptance et bout en bout.	jeter le travail par-dessus le mur vers un rôle QA fourre-tout.
Qui s'occupe de Xray ?	**Ingénieur ou spécialiste en validation** — administration, structure des plans de validation, traçabilité spécifications → validations → résultats.	Xray est l'outil de gouvernance de la validation. Il exige une compétence spécialisée en gestion de tests et traçabilité.
Qui signe le Go/No-Go ?	**CdL** prépare le dossier. **Portfolio** discute et vote. **VSE** prend la décision finale.	La décision est informée par les métriques, signaux, risques et dérogations. Le VSE assume la responsabilité finale — même quand le vote est partagé.
Qui teste le code que le dev a fait ?	**Le dev lui-même** — tests unitaires, TDD, code review et Definition of Done. **Un autre dev** contribue à la revue par les pairs.	La qualité du code est la responsabilité du développeur. La revue par les pairs est plus efficace qu'un QA qui revérifie le code après coup.
Qui teste ce que le produit doit faire ?	**Ingénieur ou spécialiste en validation** — validation fonctionnelle basée sur le SPECIFY.	C'est la validation du produit par rapport aux spécifications, pas la vérification du code. C'est le cœur du métier de la validation.
Qui teste l'intégration ?	**Dev / Arch** pour l'intégration technique : composants, APIs, interfaces internes. **Ingénieur ou spécialiste en validation** pour l'intégration système : bout en bout, multi-composants, environnements représentatifs.	L'intégration technique est proche du code. L'intégration système exige une vue d'ensemble et des environnements représentatifs.

Qui teste toutes les possibilités ?	**Personne** — tester « tout » est impossible et non souhaitable. La validation définit une stratégie basée sur le risque : quoi tester, à quel niveau et avec quelle priorité.	L'approche basée sur le risque est plus efficace que l'exhaustivité. Elle concentre l'effort sur ce qui a le plus d'impact.
Qui teste la sécurité, la performance et les autres validations spécialisées ?	**Spécialistes concernés** pour les domaines spécialisés. **Dev / Arch** applique les bonnes pratiques de base. **Validation** coordonne et intègre ces validations dans la stratégie globale.	Les validations spécialisées exigent des compétences spécialisées. Il ne faut pas tout mettre sur le dos d'un seul rôle QA fourre-tout.
Qui fait l'audit ?	**Chargé de livraison** pour l'audit de conformité processus : SDLC respecté, DoD appliquée, gating prêt. **Validation** pour l'audit de traçabilité : spécifications → validations → résultats.	L'audit doit être distingué de l'exécution. Le CdL vérifie le processus; la validation vérifie la couverture et la preuve de validation.

Recommandation : repositionner la qualité comme une pratique distribuée et visible, soutenue par des rôles catalyseurs, des validations traçables, des critères de sortie et des preuves qualité. Le rôle qualité moderne ne doit pas absorber les responsabilités des autres; il doit rendre les responsabilités explicites, applicables et mesurables.

Carte de priorisation et dépendances entre recommandations

Priorité	Recommandation	Dépend de
🔴 **1 — Urgente**	Consolidation des projets Jira par produit	—
🔴 **2 — Urgente**	Gouvernance Jira et Xray — livraison vs définition	Consolidation Jira

🟠 **3 — Haute**	Modèle de versionnement et découpage des livraisons	Gouvernance Jira/Xray
🟠 **4 — Haute**	Contrat qualité	Gouvernance Jira/Xray
🟠 **5 — Haute**	Responsabilité produit — PM, PO, CdL	Peut démarrer en parallèle
🟡 **6 — Moyenne**	Centralisation de la documentation produit	Gouvernance Jira/Xray, contrat qualité
🟡 **7 — Moyenne**	Positionnement de la qualité dans l'organisation	Contrat qualité, centre d'excellence
🟡 **8 — Moyenne**	Centre d'excellence	Responsabilité produit
🟡 **9 — Moyenne**	Gouvernance des rôles et pouvoir décisionnel	RACI, centre d'excellence
🟡 **10 — Moyenne**	Posture qualité dans les équipes de développement	Positionnement qualité, contrat qualité
🔵 **11 — En continu**	Shift left de l'acceptation	Contrat qualité, Xray
🔵 **12 — En continu**	Voix du client	Responsabilité produit
🔵 **13 — En continu**	Usage structuré de l'IA	Documentation centralisée
🔵 **14 — En continu**	Automatisation des validations	Gouvernance Xray, spécifications testables